

On the day — the operational page

Everything here is logistics rather than engineering: keys, money, deadlines, who to ask. One page, so that when something non-technical is in your way you know where to look.

Where things stand

The AgentSpec round closed on **Tuesday 15 September**. Everyone in this room came through it, and the rubric those specs were scored against — [SPEC-RUBRIC.pdf](#) — has not moved. Read it once more if you have not looked since you submitted.

What your spec said is not binding. If two days of building show you a better shape, take it, and say so in the demo. Changing your mind because you learned something is a finding, not a failure — it is the one kind of pivot we are glad to see.

Before you write any code

Three things, announced at the start of Day 1 and repeated here so nobody has to carry them in their head. All three exist so that Sunday's judging is about what you designed.

1 · Tell us what you are bringing

Reusing earlier work is allowed. We would rather you build on something real than start from an empty folder.

Initialise your repo, and make your **first commit to a file** called [PRE-EVENT-ASSETS.md](#). In it, list everything you are bringing in:

- prior code — a previous hackathon, a course project, your own weekend work
- prompts, agent definitions or evaluation sets written before today
- datasets you did not gather today
- any library beyond the obvious ones

A paragraph is enough. This is a declaration, not a defence. **Nothing on that list costs you marks. Something missing from it will.**

Then **submit your repo URL through the form** — the link is at the desk and in the group — so the judges can find your work without having to ask you for it.

2 · Commit three times on Day 1

11:00 am · 2:00 pm · 5:00 pm. Push whatever you have at that moment.

Broken code is fine. Half-finished is fine. Commit it anyway.

We are not scoring the commits, and missing one is not a penalty. The judges read what is there — it is how we see the work grow. An empty history leaves them nothing to read.

3 · Be ready for a change on Sunday morning

There may be a change to the brief at **9:00 am on Day 2**. If it comes, we will set it from where the cohort as a whole has got to by Saturday evening.

Your API key

You get a key at the registration desk beginning **sk-or-v1-**. It goes in **.env** after **OPENROUTER_API_KEY=** and nowhere else — never in code, never in a commit, never in a screenshot.

One key per	team — one shared budget, and you will feel the ceiling by lunchtime. That is deliberate
Starting allowance	\$10.00 — finishing inside it without a top-up is a design result, not thrift. Say so in the demo
Top-up	Once only and to max of \$5.00 — bring your team name and your current doctor.py output
Help desk for keys	Calvin K (student) and Rajeev Yelkur (alum) — numbers circulated separately, not in this repo

The cap is a real fence, not a warning. When your key cannot cover a request, the provider refuses it *before* running it — so you cannot accidentally overspend. A top-up is a decision someone makes at the desk; it is not something your code can reach for at three in the morning.

Reading the two errors that look identical and are not

`python scripts/doctor.py` tells you which one you have, and they need opposite responses.

what you see	what it means	what to do
<code>refused for credit (402)</code>	Your key cannot cover a request this size	Lower <code>SLICE_MAX_TOKENS</code> first — the provider reserves against it, so a big ceiling can refuse you while you still have usable credit. If that does not fix it, go to the desk
<code>rate-limited (429)</code>	Not your fault. The provider is throttling	Wait a few minutes, or switch to <code>SLICE_FALLBACK_MODEL</code> . Do not go to the desk; there is nothing they can do
<code>unreachable (HTTP 000)</code>	Network, not code	Check the wifi before you check anything you wrote
<code>key works – \$X of \$Y left</code>	Healthy	Glance at it occasionally. If it is dropping fast, something is looping

If the whole room goes down at once, it is the shared pool, not your key. Tell the desk once and carry on with fixtures or stubbed answers — that is exactly the situation recorded model responses exist for.

What you are judged on

	weight
A working agentic slice — it runs, and one step judges another step's work and sends it back	35
Evidence that real people used it — walkthroughs with people outside your team, and what you changed because of what you watched	35

	weight
Whether it helped — what that person could do afterwards that they could not before; and what you did when one of your own assumptions turned out to be wrong	20
How you worked, and how you show it — the commit rhythm, covering the ground in the demo, and how you handle questions	10

Evidence carries the same weight as the build. That is not a rounding decision — it is the whole point. It cannot be produced on the last afternoon, and it is the part most teams discover too late. If you have not put your agent in front of a stranger by Saturday evening, you are already behind on a third of the score.

And finding out that it did not help still counts. A team that tested properly, learned their idea does not work the way they thought, and changed the design because of it scores as well as a team whose first guess was right. What scores badly is never having looked hard enough to find out.

The demo

Format and timings are settled on the day. Whatever they are, the demo has to carry four things.

The problem. Who it is for, and what went wrong for them. This is context rather than a scored item of its own, but nothing after it makes sense without it.

One live run, end to end — and it must show **the step that sends work backwards**. If the loop is not visible in the run, it cannot be scored. This is the single most common way a good build loses marks.

What happened when real people used it. Names. What they did, what broke, what you changed, and whether they were better off afterwards.

Questions. Expect to be asked to *show it failing*. Thirty seconds, no warning. A team that can make their own system break, and explain why, is telling the judges something a rehearsed demo cannot.

Three rules:

- **It runs from your frozen commit.** The final commit closes before demos start, and the coordinator checks `git log`. Code that is not in the repo did not happen.
- **One slide maximum.** Slides are where a thin build hides.
- **It runs live, or you show a recording you made before the freeze and say so.** No mock screens.

A break is not a failure. A system that crashes and a team who can say exactly why beats a system that only ever runs on one rehearsed input. What costs you is not knowing what went wrong.

The two days

19 and 20 September, 9:00 to 6:00 on site each day. The venue empties at six.

Day 1 opens with the inauguration, **9:00 to 9:50** — you are in the hall for it. Building starts straight afterwards. Day 2 closes with the valedictory. Neither is optional and neither is long; the rest of both days is yours.

You are free to keep working in the evening and pick up again the next morning, but plan the work — and especially anything that needs other people in the room — around the hours you are actually together.

That last point matters most for walkthroughs. Three testers, twenty minutes each, plus someone willing to try to break it: book names and times, and book them for when the building is full.

Where to ask

about	ask
Keys, money, the desk	Calvin K or Rajeev Yelkur, at the key desk
The starter kit, or something in <code>slice/</code> behaving oddly	Rajeev Yelkur
Submission and logistics	<code>agentathon.cse@gmail.com</code>

If you find a bug in `slice/` itself, tell the desk rather than only fixing your own copy. Every team is running the same code, and you will have saved several of them in an afternoon.